C++

C++ is a general-purpose, object-oriented programming language developed by Bjarne Stroustrup in 1979. It extends the C language by adding features like

* object-oriented programming (OOP)
* Templates
* Exception handling
* standard Template Library (STL)
* Generic programming

Application

* Embedded s/m
* operating s/m
* Game development
* Robotics
* Desktop Application
* AI libraries
* Financial s/w

structure of C++

```
#include <iostream> // i/p o/p library
using namespace std; // standard namespace
int main() // Program starts here
{
  cout << "Hello"; // o/p
  return 0; // Program ends
}
```

without namespace

```
#include <iostream>
int main(){
std::cout << "welcome";
}
```

o/p
welcome

with namespace

```
#include <iostream>
int main(){ using namespace std;
int main(){
cout << "good morng";
```

o/p
good morng

next line

```
#include <iostream>
int main(){
std::cout << "welcome" << "\n";
"      "   << "good morng";
}
```

(or)

```
#include <iostream>
using namespace std;
int main(){
cout << "welcome" << endl;
"      " "good morng";
}
```

i/p from user

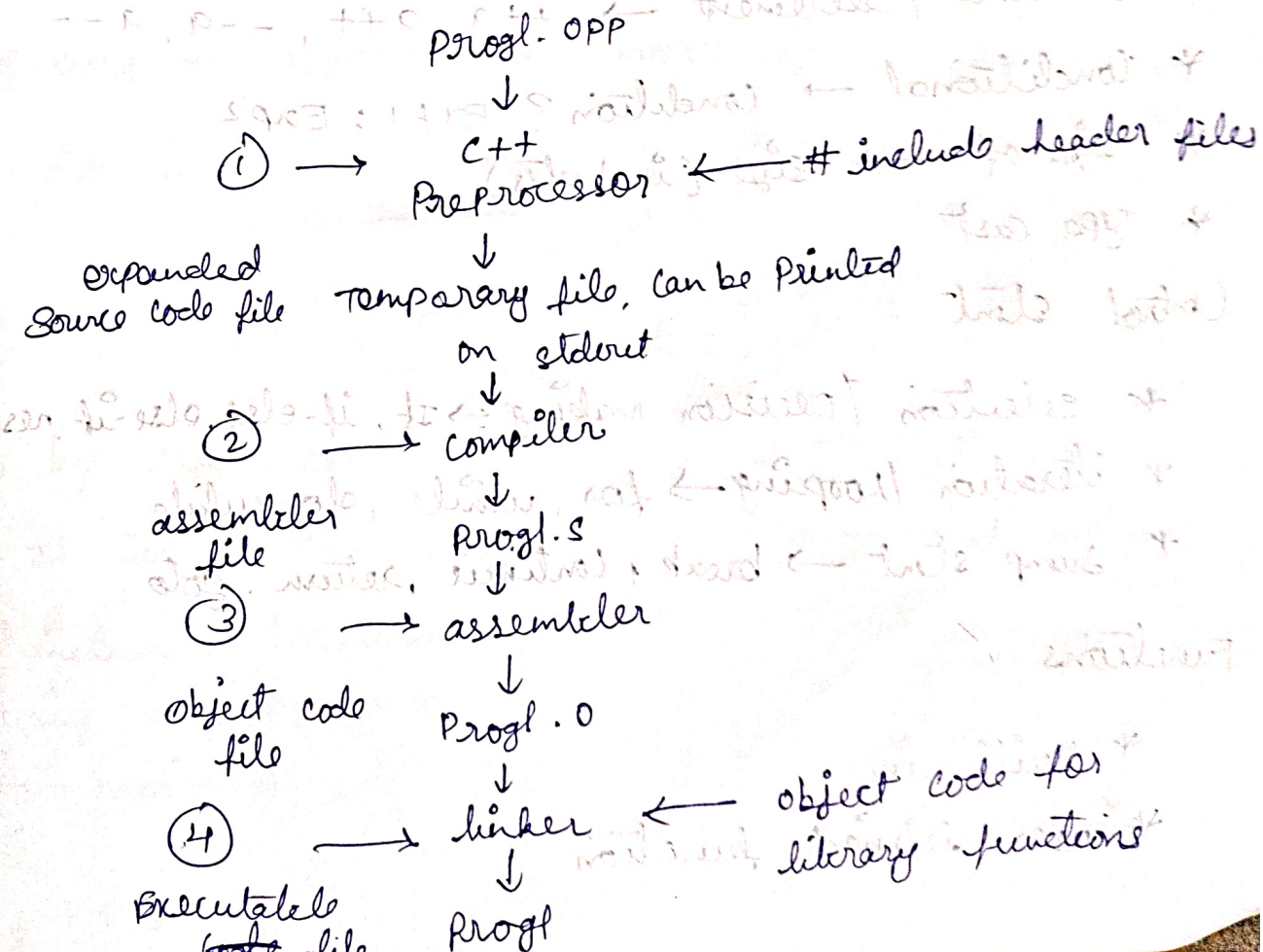
```
# include <iostream>
using namespace std;
int main ()
{
 string name;
cout << " Enter your name ";
cin >> name;
cout << " your name is " << name;
}
```

Variable

int age = 20; , float sal = 25000.5; , char grade = 'A';

bool pass = true; , string name = "Vaidhi";

Build & Git workflow Integration



Types of error

1) Syntax error 2) Runtime error

Data type

1) Fundamental (Build-in) types
   - Integer
   - Float
   - Char
   - Boolean

2) Derived types
   - Array
   - Pointer
   - Reference

operators

* Arithmetic → +, -, *, /, %, ++, --
* Relational → ==, !=, >, <, >=, <=
* Logical → &&, ||, !
* Assignment → =, +=, -=, *=, /=, %=
* Bitwise → &, |, ^, ~, <<, >>
* Increment / Decrement → ++a, a++, --a, a--
* Conditional → condition ? Exp1 : Exp2
* Size of → size (in bytes)
* Type cast

Control stmt

* selection / Decision making → if, if-else, else-if nested if switch
* iteration / Looping → for, while, do-while
* Jump stmt → break, continue, return, goto

Functions

* Built-in
* user-defined function

Syntax

```
Return type ← int calculate_sum (int num1, int num2)   ← function name, function parameters
{
    int sum = num1 + num2;
    return sum;
}
```
→ function body

Predefined

<cmath> → sqrt(x), pow(base, exp), abs(x), ceil(x), floor(x), max(a,b), min(a,b)

<string> → getline(cin,s), stoi(s), to_string(x)

<cctype> → upper, lower

<algorithm> → sort, reverse

Pass by Value

original variable not modified

```
void incre(int x)
{
    x++;
}
```

Pass by reference

It can modified

```
#include<iostream>
using namespace std;
void incre(int &x)
{
    x++;
}
int main()
{
    int a = 10;
    incre(a);
    cout<<a;
}
```

Syntax

```
Return type ← int calculate_sum (int num1, int num2)   ← function name, function parameters
{
    int sum = num1 + num2;
    return sum;
}
```
→ function body

Predefined

<cmath> → sqrt(x), pow(base, exp), abs(x), ceil(x), floor(x), max(a,b), min(a,b)

<string> → getline(cin,s), stoi(s), to_string(x)

<cctype> → upper, lower

<algorithm> → sort, reverse

Pass by Value

original variable not modified

```
void incre(int x)
{
  x++;
}
```

Pass by reference

It can modified

```
#include <iostream>
using namespace std;
void incre(int &x)
{
  x++;
}
int main()
{
  int a = 10;
  incre(a);
  cout << a;
}
```

OOPs Concept in C++

Class

* Classes
* Objects
* Encapsulation
* Abstraction
* Inheritance
* Polymorphism

classes.

It is a blueprint to create an object

Ex:

```
class classname {
private:
    // Private data member
Public:
    // Public data
    // member function
};
```

---

object

object is an instance of a class
it is a real world entities

Ex:

```
class car {
public:
  void start (){
    cout << "car start";
  }
};
int main (){
  car c; c.start();
  return 0;
}
```

Ex:

```
class phone {
  public:
  void power (){
  cout << "phone power on";
}};
int main (){
 phone p1;
 p1.power();
phone p2;
p2.power();
}
```

OOPs Concept in C++

Class

* Classes
* Objects
* Encapsulation
* Abstraction
* Inheritance
* Polymorphism

classes.

It is a blueprint to create an object

Ex:

```
class classname {
Private:
   // Private data member
Public:
   // Public data
   // member function
};
```

Object

object is an instance of a class
it is a real world entities

Ex:

```
class Car {
Public:
Void start () {
   cout << "car start";
}
};
int main () {
   car c; c.start();
   return 0;
}
```

Ex:

```
class phone {
  Public:
  Void Power () {
 cout << "phone Power on";
}};
int main () {
 phone P1;
P1.Power ();
phone P2;
P2.Power ();
}
```

Difference B/w Constructor & Destructor in C++

Constructor

* called when the object is created
* constructor accepts parameters
* can be overloaded
* can be multiple constructors

Destructor

* called when the object is destroyed or deleted
* Doesn't accept parameters
* Can't be overloaded
* there's only a single deconstructor

Ex.

```
#include <iostream>
using namespace std;
class student {
Public:
  student(){
    cout << "constructor called\n";
  }
  ~student(){
    cout << "Destructor called";
  }
};
int main(){
  student s;
  return 0;
}
```

Inheritance

One class can acquire properties & methods of another class.

* Parent class → Base class / super class.
* Child class → Derived class / sub class.

Why we use Inheritance

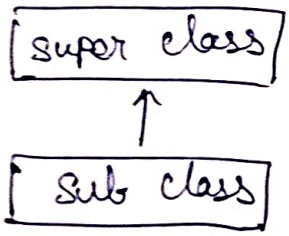
* Code reusability
* Less duplication
* Easy maintenance
* Better Organisation

Types of Inheritance

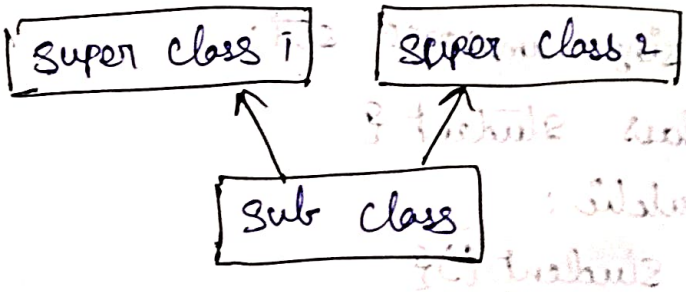
1) Single Inheritance
2) Multiple Inheritance
3) Multilevel "
4) Hierarchical "
5) Hybrid "

Types of Inheritance

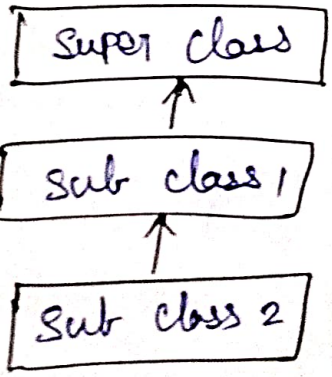
Single Inheritance



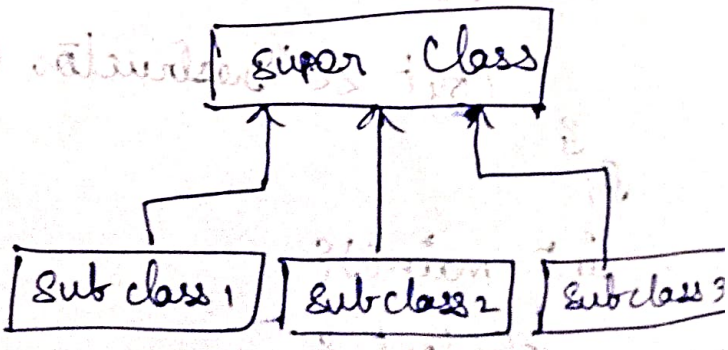
Multiple Inheritance



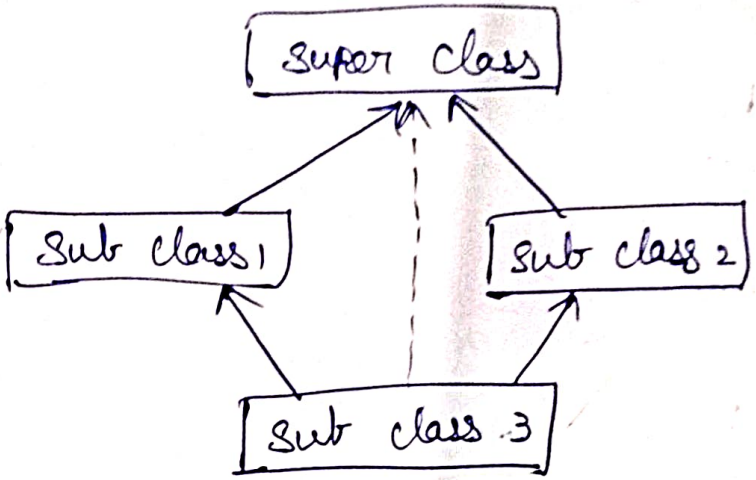
Multilevel Inheritance



Hierarchical Inheritance



Hybrid Inheritance



1) single Inheritance

syntax :-

parent class
↓
child class

```
class Base
{
  // members
};
class Derived : public Base
{
  // members
};
```

Ex)

```
class Vehicle
{
public:
  void move()
  {
    cout << " Vehicle is moving " << endl;
  }
};
class car : public Vehicle
{
public:
  void horn()
  {
    cout << "Car horn: BeepBeep!" << endl;
  }
};
int main() {
  Car myCar; myCar.move();
  myCar.horn();
}
```

Inheritance

2) multiple Inheritance

one child inherits from multiple parents

Syntax:

```
Class parent 1:
  pass
Class parent 2:
  pass
class child (Parent 1, Parent 2);
  pass
```

Ex:

```
class Camera
{
 Public:
   Void click ()
   {
    cout << "Taking photo" << endl;
   }
};
 class music player
 {
  Public:
    Void play_music ()
    {
     Cout << "playing music" << endl;
    }
 };
class Smartphone : Public Camera, Public music player
 {
 };
```

```
int main()
{
  Smartphone S1;
  S1.click();
  S1.play_music();
  return 0;
}
```

3) Multilevel Inheritance

Inheritance chain continues level by level

Syntax

```
class A
{
};
class B : public A
{
};
class C : public B
{
};
```

Ex:

```
class Person
{
 public:
   void displayName()
   {
    cout << "Name: John" << endl;
   }
};
class Employee : public Person
{
 public:
   void displaySalary()
   {
    cout << "Salary: 50000" << endl;
   }
};
class Manager : public Employee
{public:
  void displayDepartment()
  {
    cout << "Department: IT" << endl;
  }
};
int main(){
 Manager m;
m.displayName(); m.displaySalary(); m.displayDepartment();
```

4) Hierarchical Inheritance

Multiple child classes inherit from one parent

syntax:

```
class parent{
};
class Derived1: Public Base{
};
class Derived2: Public Base{
};
```

Program:

```
class shape{
Public:
  void display(){
  cout<<" this is a shape"<<endl;
  }
};
class circle: Public shape{
Public:
  void displaycircle(){
  cout<<" this is circle"<<endl;
  }
};
class square: Public shape{
Public:
  void displaysquare(){
  cout<<" this is square"<<endl;
  }
};
```

5) Hybrid Inheritance

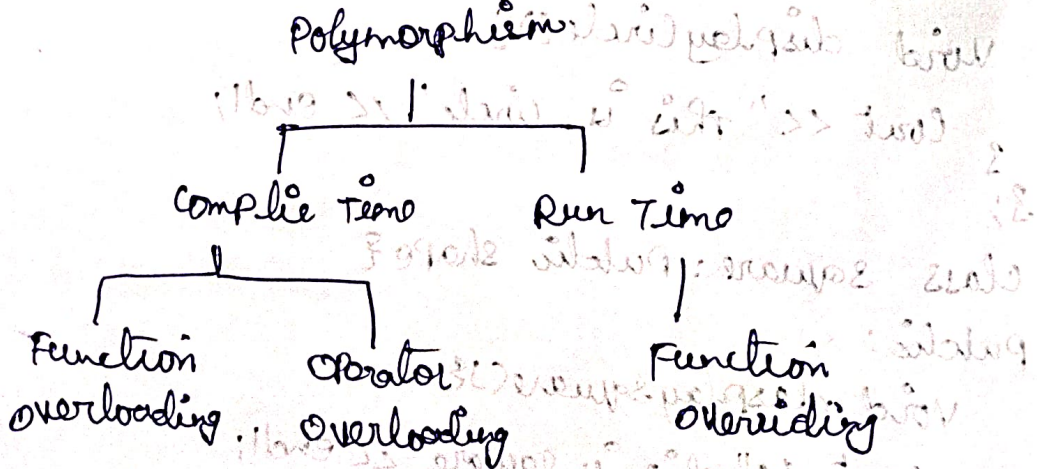
Combination of two or more inheritance types

Program :-

```
Class Base {
};
Class Derived 1: Public Base {
};
Class Derived 2: Public Base {
};
class Final Derived : public Derived 1, public Derived 2 {
};
```

Polymorphism :-

* Polymorphism means: "one thing, many forms"
* In Python, the same method/function/operator can behave differently for different objects



1) method / function Overriding

A derived class provides its own implementation of a function that already exists in the base class

```
class Animal
{
  public:
    virtual void Sound()
    {
      cout << "Animal makes a sound" << endl;
    }
};
  class Dog: public Animal
  {
    public:
      void Sound()
      {
        cout << "Dog barks" << endl;
      }
    };
int main()
{
  Animal *a;
  Dog d;
  a = &d;
  a -> Sound();
  return 0;
}
```

2) method / function overloading

Function overloading means having multiple functions with the same name but different parameters

```
class calculator
{
  Public:
   int add(int a, int b)
   {
     Return a+b;
   }
   int add (int a, int b, int c)
   {
     return a+b+c;
   }
};
```

```
int main()
{
   calculator c;
cout << c.add (5,10)<<
cout << c.add(5,10,15);
return 0;
}
```

3) operator overloading

operator overloading allows an operator to work differently for user-defined objects

```
class Demo
{
  Public:
   int value;
   Demo (int v)
   {
     value = v;
   }
   Demo operator +(Demo obj)
   {
     Return Demo(value + obj.value);
   }
};
```

```
int main()
{
  Demo d1(10), d2(20)
  Demo d3 = d1+d2;
cout << d3.value;
return 0;
}
```

```
class calculator
{
  Public:
   int add(int a, int b)
   {
     Return a+b;
   }
   int add (int a, int b, int c)
   {
     return a+b+c;
   }
};
```

```
int main()
{
  calculator c;
  cout << c.add(5,10)<<
  cout << c.add(5,10,15);
  return 0;
}
```

3) operator overloading

operator overaloading allows an operator to work differently for user-defined objects

```
class Demo
{
  Public:
   int value;
   Demo(int v)
   {
     value = v;
   }
   Demo operator +(Demo obj)
   {
     Return Demo(value + obj.value);
   }
};
```

```
int main()
{
  Demo d1(10), d2(20);
  Demo d3 = d1 + d2;
  cout << d3.value;
  return 0;
}
```

Abstraction

Abstraction is an object-oriented programming (OOP) concept that simplifies complex system by hiding internal implementation details & exposing only the essential features of an object.

```
class Base
{
 Public:
   virtual void display() = 0;
 };
 class Derived : public Base
 {
   Public:
     void display()
     {
       //
     }
 };
```

Ex:

```
#include <iostream>
using namespace std;
class Communication
{
 Public:
   virtual void sendData() = 0;
 };

 class UART : public Communication
 {
  Public:
```

```
void SendData()
{
  Cout << "Sending Data Through UART" << endl;
}
};
class SPI : Public Communication
{
 Public:
  void SendData()
  {
   Count << "Sending Data Through SPI" << endl;;
  }
};
  int main()
{
  UART U;
  SPI S;
  U.SendData();
  S.SendData();
 return 0;
}
```

Encapsulation

Encapsulation is a core object-Oriented Programming Principle that bundles data & the methods that operate on that data into a single unit, usually a class.

Its Primary goal is data hiding, which prevents direct external access to an objects internal state to avoid accidental modification.

Class

Variable	methods

Encapsulation

Public → name → anywhere

Protected → " → Inside class & child class

Private → " → Inside class only

accessible from anywhere

```
#include <iostream>
using namespace std;
class student
{
  Public
    string name;
    void display()
    {
    cout << "Name :" << name << endl;
  }
};
```

```
int main()
{
 student s;
 s.name = "Ravi";
 s. display();
 return 0;
}
```

2) Protected members

```
class Employee
{
 Protected:
   int salary;
};
 class Manager : Public Employee
{
```

Class

Variable	methods

Encapsulation

Public → name → anywhere

Protected → " → Inside class & child class

Private → " → Inside class only

accessible from anywhere

```
#include <iostream>
using namespace std;
class student
{
  Public
    string name;
    void display()
    {
    cout << "Name :" << name << endl;
  }
};
```

```
int main()
{
 student s;
 s.name = "Ravi";
 s. display();
 return 0;
}
```

2) Protected members

```
class Employee
{
 Protected:
   int salary;
};
class Manager : Public Employee
{
```

```
Public:
  void setSalary(int s)
  {
    salary = s;
  }
  void display()
  {
    cout << "Manager Salary:" << salary << endl;
  }
};
```

```
int main()
{
  Manager m;
  m.setSalary(60000);
  m.display();
  return 0;
}
```

3) Private members.

```
Class BankAccount
{
 Private:
  int balance;
 Public:
  void deposit(int amount)
  {
    balance = amount;
  }
  void showBalance()
  {
  cout << "Balance": << balance
       << endl; } };
```

```
int main()
{
 BankAccount b;
 b.deposite(10000);
 b.ShowBalance();
 return 0;
}
```

Access Specifiers in C++

	Access within class	Access from outside class	Access in Derived class
1) Private	yes	NO	NO
2) Protected	yes	NO	yes
3) Public	yes	yes	yes

Ex:

```
#include <iostream>
using namespace std;
class student{
 private:
   int rollNo; //Private data
   float marks; //
Public:
  //setter function
void setData(int r, float m){
   rollNo = r;
   marks = m;
}
//getter function
void display(){
 cout<<"Roll NO:"<<rollno<<endl;
  "  "  "Marks :"<< marks <<  ;
}};
int main(){
  student s;
 s.setData(101, 95.5);
 s.display();
//s.rollNO=200; //error! Private member
 return 0;
}
```

O/P

Roll NO: 101

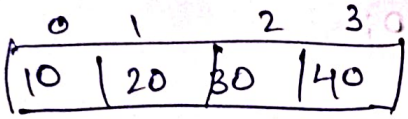
Mark : 95.5

C++ STL (Standard Template Library)

The Standard Template Library in C++ is a powerful collection of template classes & function that Provide ready-made data structure & algorithms

- Containers → vector, array, list, deque
  Data structures that store collection of objects
- Algorithms

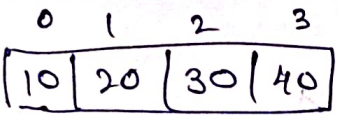
1) vector → dynamic array



A dynamic array that can resize automatically

```
std :: vector <int> V= {10, 20, 30, 40};
```

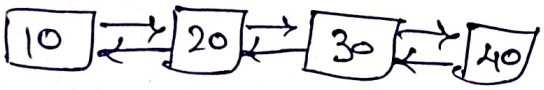
2) array → fixed array



A fixed size array with size known at compile time

```
std :: array <int, 4> a = {10, 20, 30, 40};
```

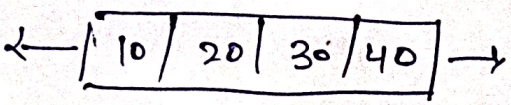
3) List → linked list



A doubly linked list. Efficient insertion/deletion

```
std :: list <int> lst = {10,20,30,40};
```

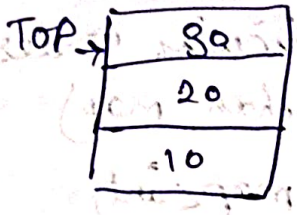
4) deque → both ends



Double ended queue. Insert/delete from both front & back

std :: deque<int> dq = {10, 20, 30, 40};

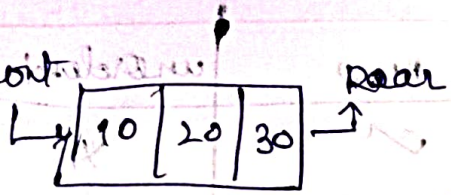
5) stack → LIFO



→ Last in First out, insert & remove from top

std :: stack <int> st;
st.push(10);

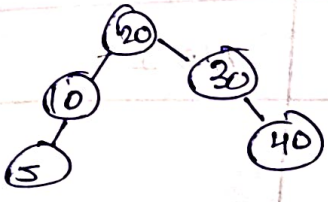
6) queue → FIFO



First in 1st out, insert at rear, remove from front.

std :: queue <int> q;
q.push(10);

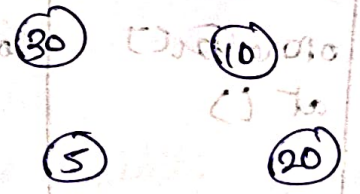
7) set → unique + sorted



stores unique elements in sorted order

std :: set <int> s = {10, 20, 20, 30};
// {5, 10, 20, 30}

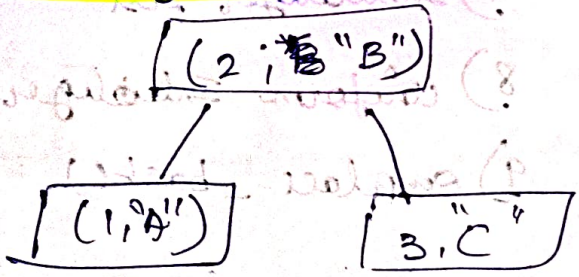
8) unordered_set → unique + unsorted



stores unique elements in no particular order. (Faster than set)

std :: unordered_set <int> s = {10, 20, 20, 30, 5};

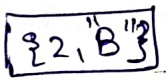
9) map → key + value + sorted

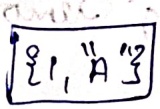


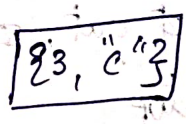
stores key-value pairs in sorted order by key.

std :: map<int, string> m;
m[1] = "A";

unordered_map → key + value + unsorted







stores Key_Value pairs in no particular order (faster than map)

```
std::unordered_map<int, string> m;
m[1] = "A";
```

Feature	vector	list	set	map	unordered_map
Ordered	Yes	✓	✓	✓	X
mutable/ immutable	Mu	MU	Immu	Immu	Immu
Duplicates	✓	~~No ordering~~ ✓	Not A	Not A	Not A
Indexing/ Access	✓	NO I	NO I	NO I	NO I
Common/ operation methods	push_back() ~~push~~ pop_back() insert() erase() size() empty() operator[]	push_front() begin(), end()	find() count()	operator[] at()	count()

Modern C++ Features: C++11 & Beyond – Simpler, Safer, Fast

1) auto keyword
2) Range-Based for loop
3) nullptr
5) Lambda Expressions
6) Smart pointers
7) Initializer List
8) Uniform Initialization
9) emplace_back()

unordered_map → key + value + unsorted

{2, "B"}

{1, "A"} {3, "C"2}

stores key-value pairs in no particular order (faster than map)

```
std::unordered_map<int, string> m;
m[1] = "A";
```

Feature	vector	list	set	map	unordered_map
ordered	yes	✓	✓	✓	X
mutable/ immutable	Mu	Mu	Immu	Immu	Immu
Duplicates	✓	~~No ordering~~ ✓	Not A	Not A	Not A
Indexing/ Access	✓	NO I	NO I	NO I	NO I
Common operation methods	push_back() ~~push~~ pop_back() insert() erase() size() empty() operator[]	push_front() begin(), end()	find() count()	operator[] at()	count()

Modern C++ Features : C++11 & Beyond - Simpler, Safer, Faster

1) auto keyword
2) Range-based for loop
3) nullptr
5) Lambda Expressions
6) Smart Pointers
7) Initializer List
8) Uniform Initialization
9) emplace_back()

C++ Templates : Generic programming

functions

* A ~~feature~~ feature allowing functions & classes to operate with generic types. The compiler generates a specific version of the template for each used type.

syntax

Function template

```
template < typename T>
T add ( T a, T b){
   return a+b;
}
add <int> (5, 10);
add <double> (3.5, 2.1);
```

Class Template

```
template < typename T >
class Box {
  T value;
public:
  void set (T v) { value = v; }
  T get () { return value; }
};
Box <int> intBox;
Box <std::string> strBox;
```

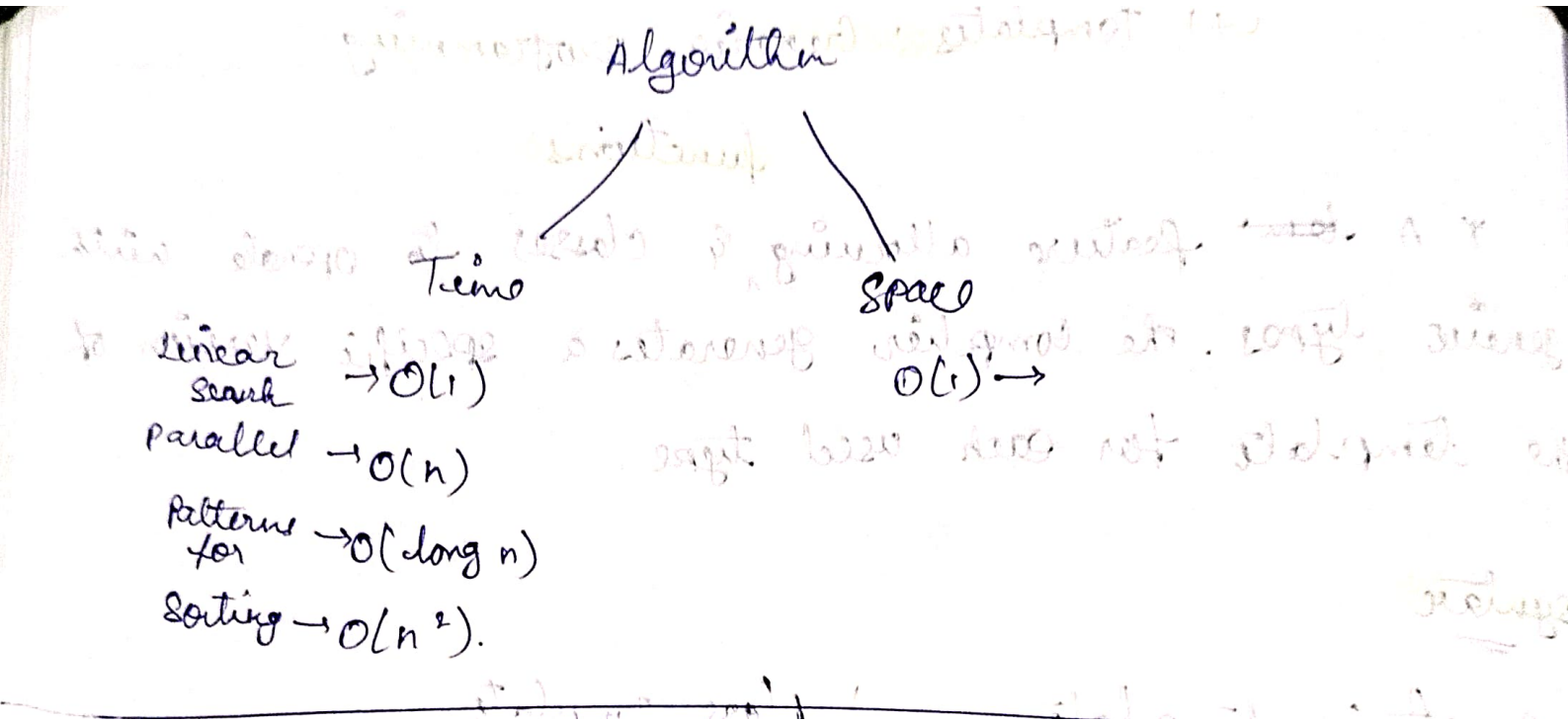
~~Algorithms~~

C++ concurrency: thread-based parallelism Guide

The ability of a program to execute multiple tasks that make progress during the same period, mainly achieved using thread

→ single - Perform the ~~sing~~ single task at a time

→ multiple thread → perform multiple task at a same time

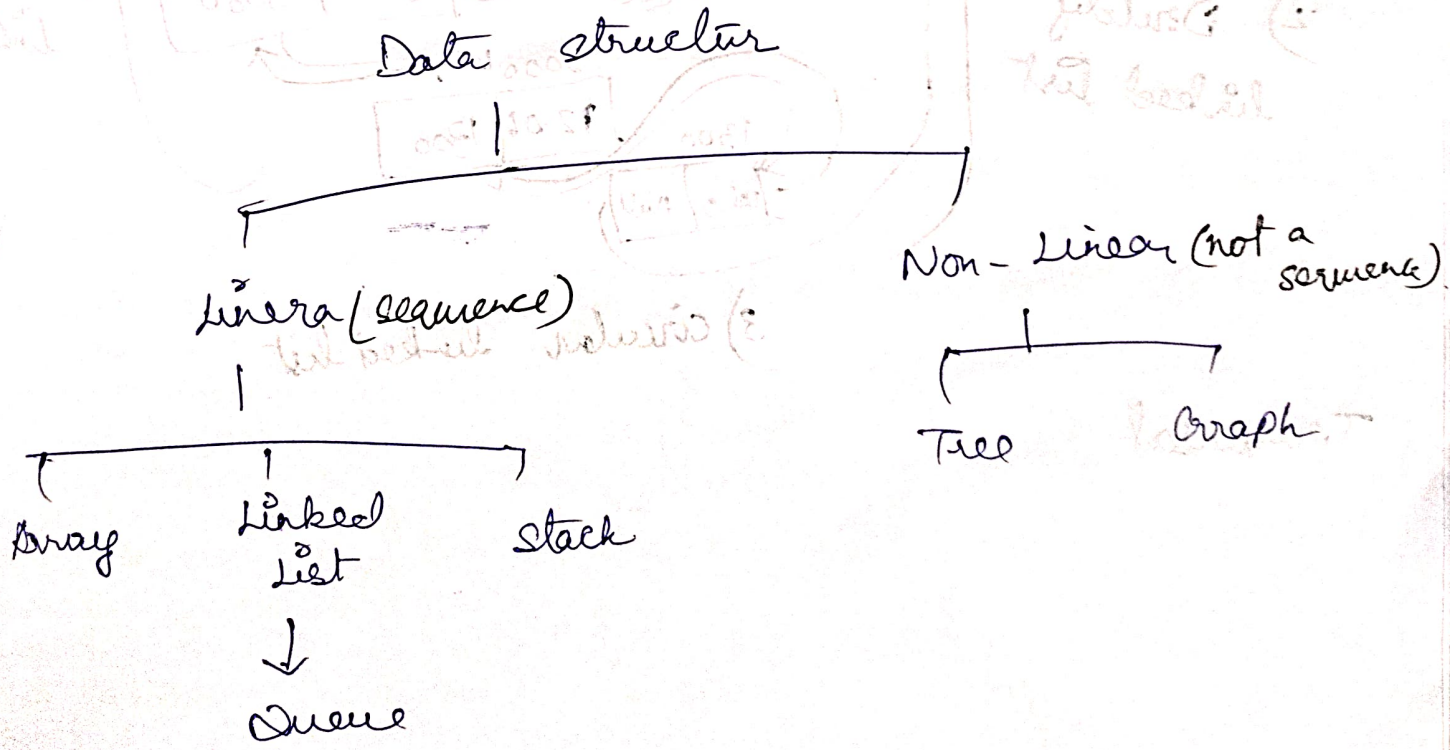

Algorithm
Time
Space
Linear search → O(1)
Parallel → O(n)
Patterns for → O(log n)
Sorting → O(n²).
O(1) →

DSA - (Data structures & Algorithms)

Data structure → ways to store & Organize data

- ↳ Array
- ↳ linked list
- ↳ Stack
- ↳ Queue
- ↳ Hash table

Algorithms → step by step instructions to solve a problem



Array disadvantages → fixed size and shifting required but compared to Array list. Array is the fast because of homogenous.

ArrayList. (Dynamic Array)

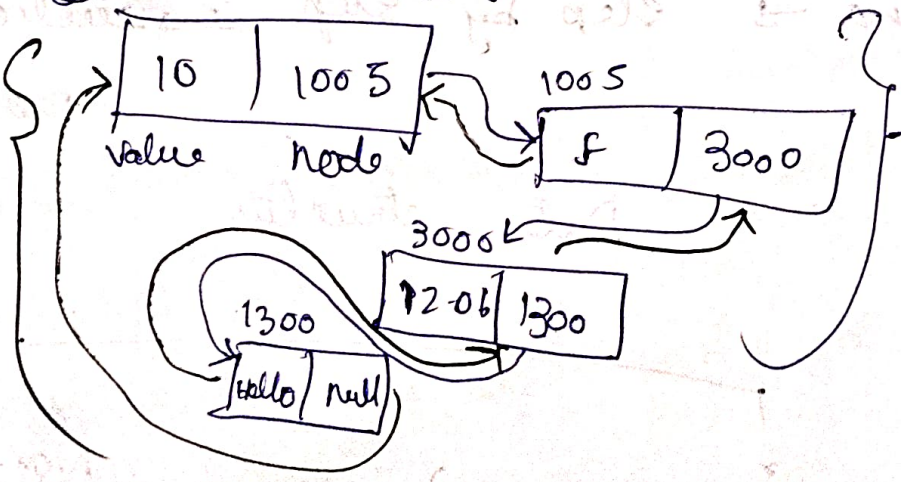
* is a part of Java collections Framework
* we can store different datatypes and it is not a fixed size memory

Java syntax

```
ArrayList<Object> a = new ArrayList<Object>();
a.add(10);
('f');
(12.06);
("Hello");
```

Linked List-

It is a linear data structure consisting of nodes.
It is also stored in two parts like value & node
It will linked based on address



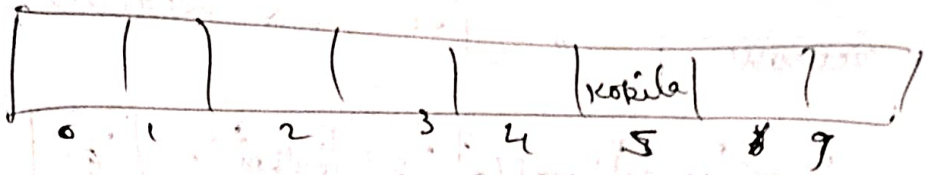
2) Doubly linked list

1) singly linked list

3) circular linked list

HashSet

It is a collection stores uniques elements
It does not maintain



Kokila

K – 107
O – 111
K – 107
I – 105
L – 108
A – 97

635

635%10 => 5

* It does not allow the duplicates
* It support the index position

TreeSet :-

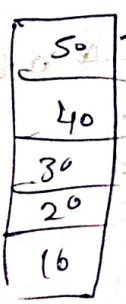
It will allows to access in the ascending order and does not allow the multiple data types it does not allow null values.

Peek → It will retrie the 1st element but does not remove

Poll → It will retrive 1st element & remove

Stack

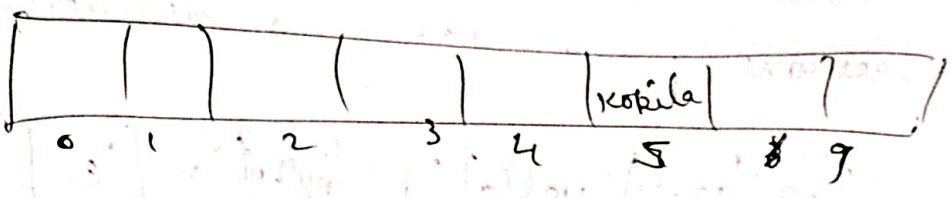
It is a linear data structure that LOFI (or) like



HashSet

It is a collection stores uniques elements

It does not maintain



Kokila

k – 107
o – 111
k – 107
i – 105
l – 108
a – 97

635

635%10 => 5

* It does not allow the duplicates
* It support the index position

TreeSet :-

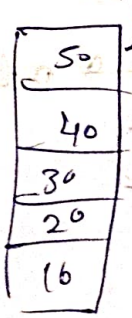
It will allows to access in the ascending order and does not allow the multiple data types it does not allow null values.

Peek → It will retrie the 1st element but does not remove

Poll → It will retrive 1st element & remove

Stack

It is a linear data structure that LOFI (or) FILO

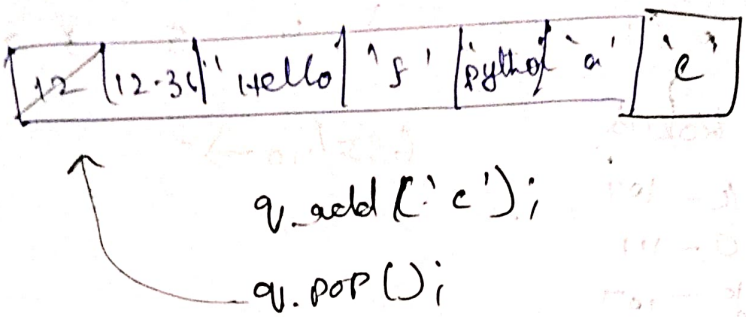


Queue :-

FIFO

elements adds at the last → enqueue

" remove " " 1st → dequeue



1) Priority Queue 2) Dequeue (Double ended queue)

without DSA

```
int a = 10, b = 20;
  int c = a+b;
Printf ("%d\n", c);
```

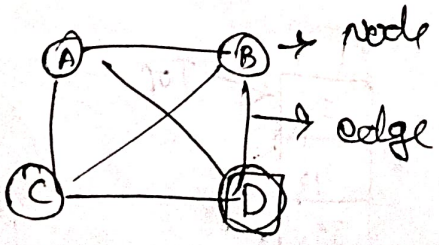
O(1)

with DSA

```
int a[] = {10, 20};
 int sum = 0;
for(int i = 0; i < a.leng
{
  sum += a[i];   O(n
}
Printf ("%d", su
```

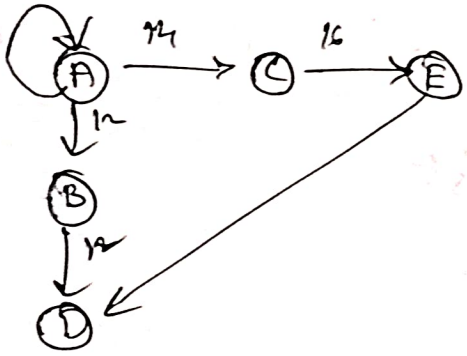
Graph

It is a non-linear data structure used to represent relationship (or) connection b/w object



Types:

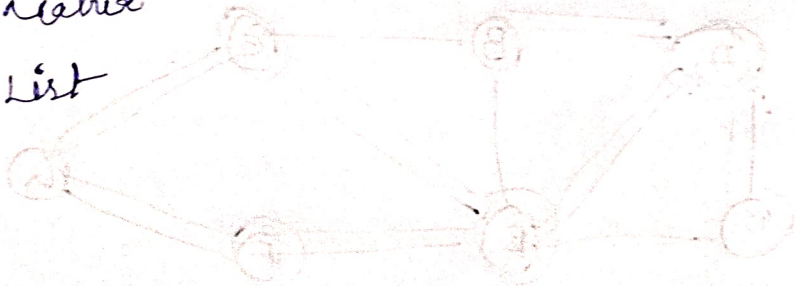
1) Directed Graph
2) weighted "
3) unweighted "
4) connected "
5) Disconnected "
6) cyclic "
7) Acyclic "
8) Directed Acyclic "
9) complete "
10) Bipartite "
11) Tree "

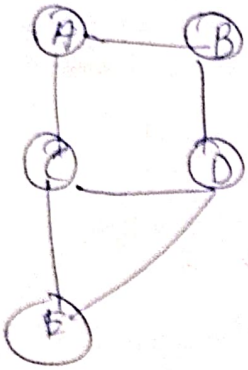


1) undirected Graph - Facebook / LinkedIn
2) Directed " - Insta / Twitter
3) weighted " - Google map
4) unweighted - ~~task scheduling~~ Network connectivity
5) DAG - Task scheduling
6) Complete " - Fully connected n/w
7) Bipartite - Job portal.

Graph representation

1) Adjacency matrix
2) " List





Adjacency matrix

	A	B	C	D	E
A	0	1	0	0	1
B	0	1	0	1	1
C	0	1	0	1	0
D	0	0	1	1	0
F	0	1	1	1	0

Adjacency list

A → B, C

B → A, D

C → A, D, E

D → B, C, E

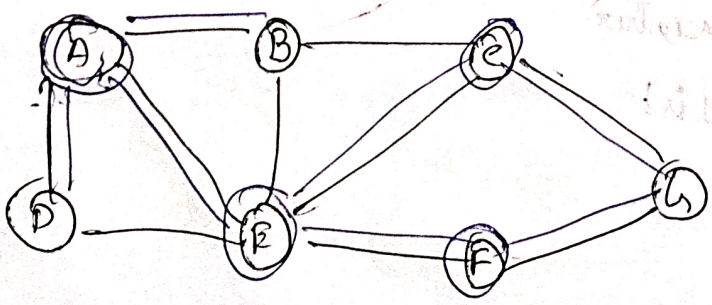
E → D, C

Graph Traversal

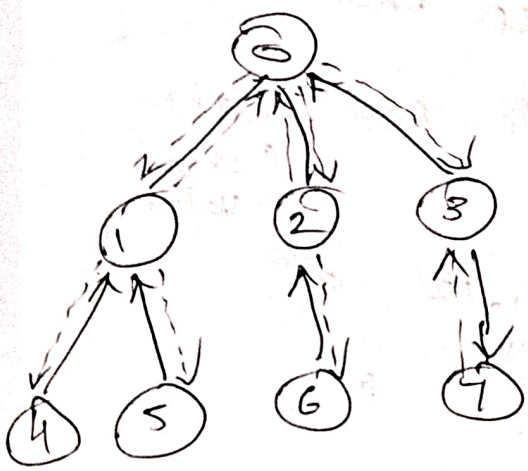
BFS.
Breadth 1st search

DFS
Deadth 1st search

BFS → It stored in queue.

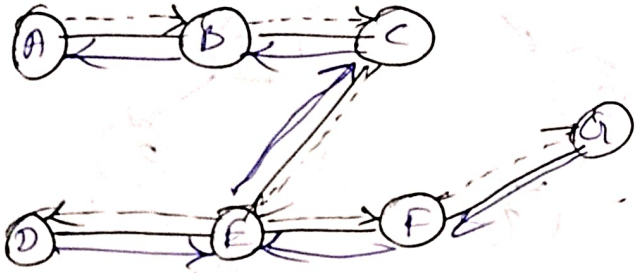


DFS → Depth 1st search



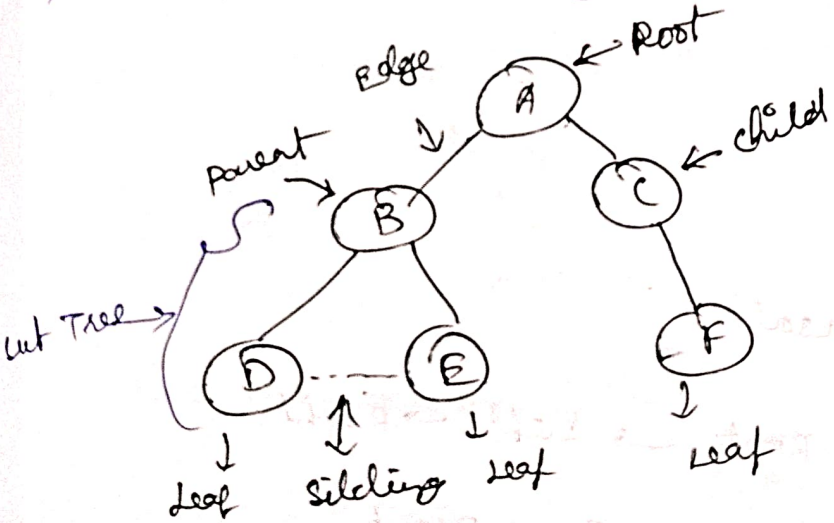
It stored in stack

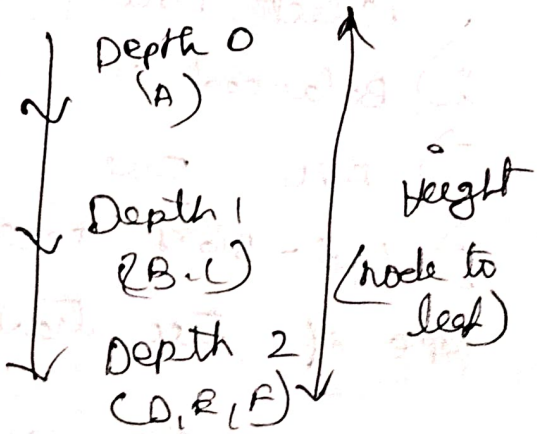
It will goes to the depth and go back to the node with back traking



Tree Data structure

Tree Graph → level by level

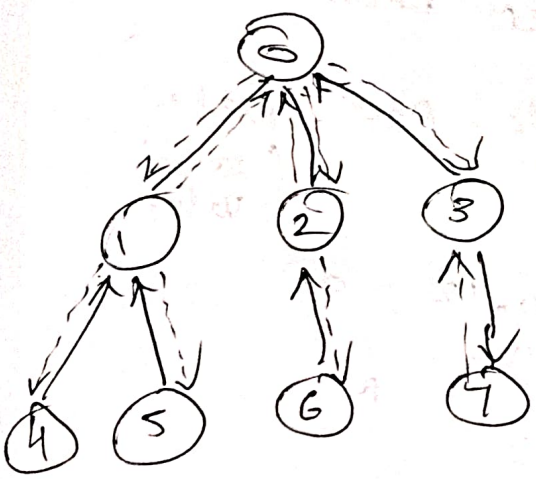




→ webpage layout

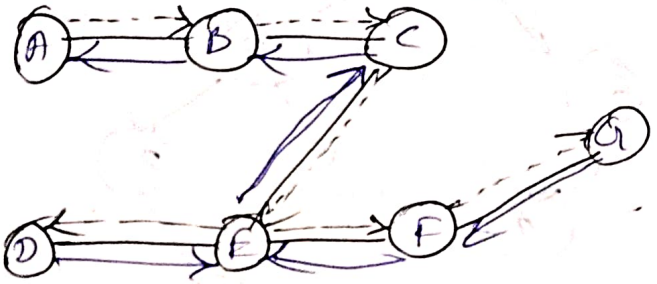
Tree Data structure → Database

DFS → Depth 1st search



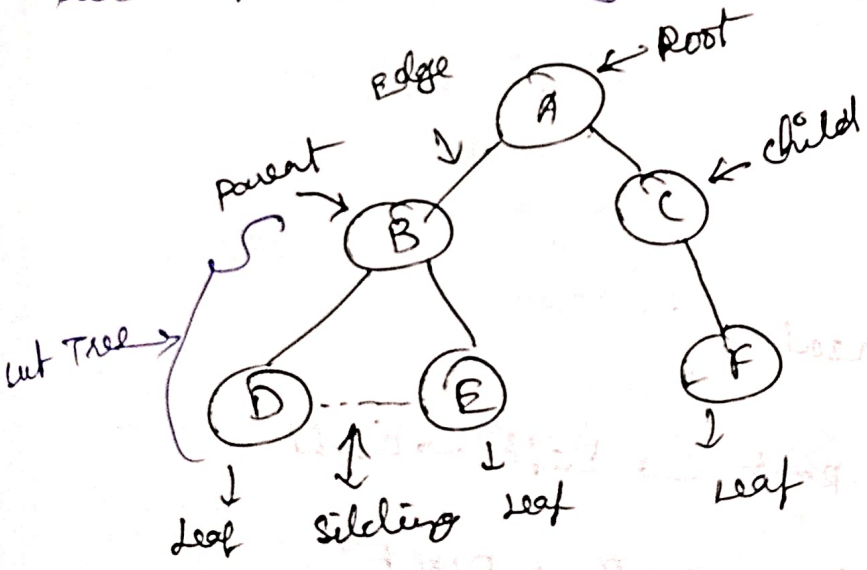
It stored in stack

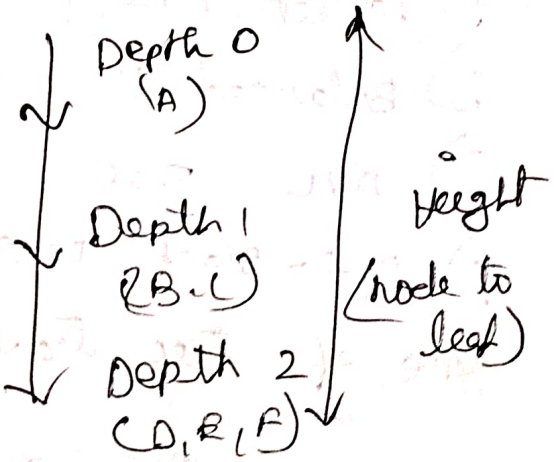
It will goes to the depth and go back to the node with back tracking



Tree Data structure

Tree Graph → Level by Level





→ Webpage layout

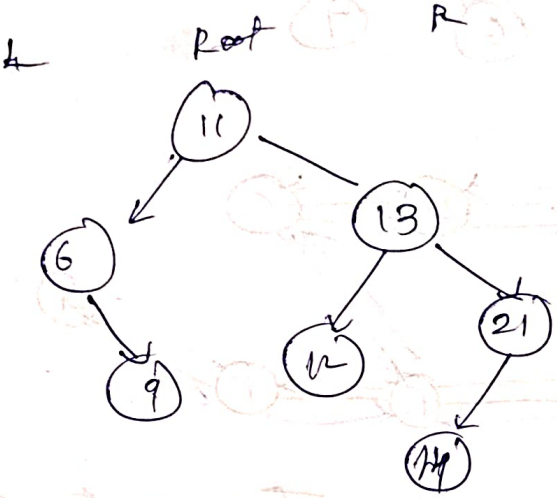
Tree Data structure → Database

Type

1) Binary Tree → almost 2 tree children

2) Binary search Tree → left side - Root - Right
(Small value) (large value)

11, 6, 13, 21, 14, 12, 9



3) Full

4)

5) Perfect Binary Tree

6) Balanced " "

7) AVL Tree

8) Red - Black Tree

Type of tree traversal

Preorder → Root → Left → Right

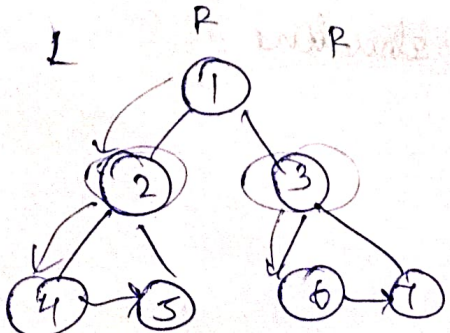
Inorder - L → R → Right

Post order - L → Right → Root

Level Order - Level by Level

preorder L R R

R → L → Ri



→ 1, 2, 4, 5, 3, 6, 7

Inorder → L → R → Ri

4, 2, 5, 1, 6, 3, 7

Postorder → L → Ri - R

4, 5, 2, 6, 7, 3, 1

searching Techniques

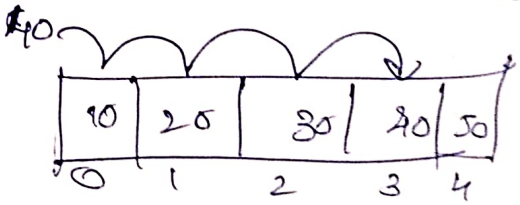
Searching is used to find whether a particular element exists in a collection of data & if it exists, where it is located.

1) Linear search → If the data is not order we use this (unsorted & sorted)
2) Binary " → If the data is order then we use this only (sorted)

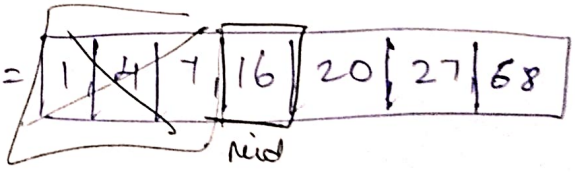
Linear → check element one by one until found the target

Binary → Repeatedly divides the search range in half

Linear →

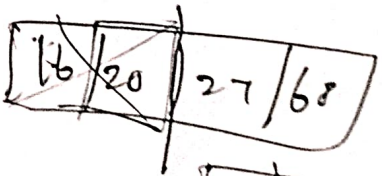


Binary → a =

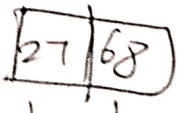


Key = 27

16 < 27



20 < 27



27 == 27

68 > 27



Sorting techniques

1) Insertion sort → for loop using
2) selection " → while loop using
3) Bubble sort → nested loop using
4) Quick sort → similar like Binar Search
5) merge " → It divides the 2 splites finally

swap the
based on
order